UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto “CodeBreaker - Aprende Lenguajes de Programación “

Curso: PATRONES DE SOFTWARE

Docente: ING. PATRICK JOSE CUADROS QUIROGA

Integrantes:

Rosas Chambilla, Jefferson	***2021072618***
Challo Coaquera Alexsander Wilson	***2017057578***
Huaman Rivera, Roberto Carlos	**2021071077**

Tacna – Perú

2026

CodeBreaker - Aprende Lenguajes de Programación
Documento de Visión

Versión *{1.0}*

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	MPV	ELV	ARV	10/04/2026	Versión Original

ÍNDICE GENERAL

1. Introducción.....6
1.1. Propósito.....6
1.2. Alcance.....7
1.3. Definiciones, Siglas y Abreviaturas.....8
1.4. Referencias.....9
1.5. Visión General.....10
2. Posicionamiento.....11
2.1. Oportunidad de negocio/educativo.....11
2.2. Definición del problema.....13
3. Descripción de los interesados y usuarios.....14
3.1. Resumen de los interesados.....14
3.2. Resumen de los usuarios.....15
3.3. Entorno de usuario.....16
3.4. Perfiles de los interesados.....17
I. Autoridad Académica (Facultad de Ingeniería - UPT).....17
II. Administración de Infraestructura Tecnológica.....17
III. Equipo de Desarrollo (UPT).....18
3.5. Perfiles de los Usuarios.....18
3.6. Necesidades de los interesados y usuarios.....20
3.6.1. Necesidades.....20
3.6.2. Requerimientos.....22
3.6.3. Requerimientos NO Funcionales.....23
4. Vista General del Producto.....25
4.1. Perspectiva del producto.....25
4.2. Resumen de capacidades.....26
4.3. Suposiciones y dependencias.....27
4.4. Costos y precios.....29
4.5. Licenciamiento e instalación.....32
5. Características del producto.....33
6. Restricciones.....35
7. Rangos de calidad.....36
8. Precedencia y Prioridad.....37
9. Otros requerimientos del producto.....38
a. Estándares legales.....38
b. Estándares de comunicación.....39
c. Estándares de cumplimiento de la plataforma.....40
d. Estándares de calidad y seguridad.....41

CONCLUSIONES.....43
RECOMENDACIONES..... 44
BIBLIOGRAFÍA.....45
WEBGRAFÍA.....45

Documento de Visión

CodeBreaker - Aprende Lenguajes de Programación

1. Introducción

El presente documento constituye la base estratégica y técnica para el desarrollo de CodeBreaker - Aprende Lenguajes de Programación. En el contexto académico actual de la enseñanza de la computación, el aprendizaje de lenguajes como Python y PHP a menudo se ve obstaculizado por la complejidad técnica inicial de configurar entornos locales de desarrollo. Este flujo tradicional depende de la instalación de compiladores, intérpretes y editores pesados que no siempre son compatibles con el hardware del estudiante.

Esta metodología presenta deficiencias críticas: una alta barrera de entrada para principiantes, falta de uniformidad en las herramientas de práctica y una desconexión entre la teoría y la ejecución inmediata. Ante este escenario, el proyecto propone una transición hacia la educación tecnológica moderna, implementando una plataforma web robusta bajo una arquitectura modular y escalable. La solución no solo centraliza el material educativo, sino que garantiza un entorno de experimentación seguro mediante el uso de Sandboxing (Docker/APIs), permitiendo la ejecución de código en tiempo real sin comprometer la infraestructura del sistema.

CodeBreaker se alinea con las tendencias de gamificación e interactividad, integrando sistemas de progresión (XP y logros) y cumpliendo con estándares de calidad ISO/IEC 25010 y accesibilidad WCAG 2.1. Con este enfoque, el proyecto busca democratizar el acceso al conocimiento técnico, optimizando la curva de aprendizaje y asegurando la persistencia de datos y el seguimiento del progreso en la nube.

1.1. Propósito

El propósito de este proyecto es optimizar el proceso de enseñanza-aprendizaje de la programación mediante el diseño e implementación de una plataforma web de tipo SPA (Single Page Application) denominada CodeBreaker. El sistema está diseñado para eliminar las barreras de entrada técnicas, como la compleja configuración de entornos locales, intérpretes y compiladores, proporcionando una solución integral accesible desde cualquier navegador moderno.

El objetivo final es transformar la experiencia formativa del estudiante mediante una arquitectura basada en React.js y Node.js, que permite la ejecución de código en tiempo real de forma segura a través de la API de Piston. Gracias a la integración del motor Monaco Editor y un sistema de gamificación con persistencia en MySQL, se busca reducir drásticamente la curva de aprendizaje, garantizando el cumplimiento de los estándares de calidad de software (ISO/IEC 25010) y aumentando la retención del conocimiento mediante un entorno visualmente dinámico y altamente interactivo.

1.2. Alcance

El sistema cubre el ciclo de vida completo del proceso de aprendizaje interactivo, desde el registro del estudiante hasta la validación de competencias técnicas. El alcance comprende:

- Módulo de Gestión de Usuarios y Perfil: Registro e inicio de sesión seguro basado en JWT (JSON Web Tokens), visualización de perfil con métricas de progreso, personalización de avatar y estadísticas personales almacenadas en MySQL.
- Módulo de Editor de Código Integrado: Interfaz profesional utilizando Monaco Editor, con soporte nativo para resaltado de sintaxis, autocompletado y manejo de errores en tiempo real para los lenguajes Python y PHP.
- Módulo de Ejecución vía Piston API: Procesamiento de scripts en entornos externos aislados y seguros, con gestión de timeouts y captura de salida (stdout/stderr) para mostrar resultados inmediatos en la interfaz de React.
- Módulo de Gamificación y Experiencia de Usuario: Sistema de asignación de XP, desbloqueo secuencial de niveles y transiciones visuales fluidas gestionadas con Framer Motion para reforzar el feedback positivo.
- Módulo de Persistencia y Sincronización: Guardado automático del estado de los ejercicios en el backend de Node.js, permitiendo al usuario retomar su progreso desde cualquier dispositivo de forma transparente.
- Módulo de Administración de Contenido: Panel administrativo para la configuración de retos, gestión de la base de datos de ejercicios y definición de las validaciones lógicas (entradas y salidas esperadas).

Fuera del Alcance: Quedan fuera del alcance la enseñanza de lenguajes de programación distintos a Python y PHP en esta fase inicial, el desarrollo de un entorno de producción profesional (enfocándose estrictamente en el ámbito pedagógico), y la integración con plataformas LMS externas (como Moodle o Canvas).

1.3. Definiciones, Siglas y Abreviaturas

Para asegurar una comprensión uniforme de los componentes de CodeBreaker, se definen los siguientes términos clave:

- SPA (Single Page Application): Aplicación web que carga una sola página HTML y actualiza dinámicamente el contenido a medida que el usuario interactúa, proporcionando una experiencia fluida sin recargas de página (implementado con React.js).

- Monaco Editor: El motor de edición de código de código abierto que alimenta a Visual Studio Code, utilizado en la plataforma para proporcionar resaltado de sintaxis, autocompletado y una interfaz de desarrollo profesional.
- Piston API: Motor de ejecución de código de alto rendimiento que permite procesar múltiples lenguajes de programación en entornos aislados y seguros, devolviendo los resultados en milisegundos.
- Sandbox (Entorno de Ejecución Segura): Entorno virtualizado y aislado donde se ejecuta el código enviado por el estudiante, evitando que scripts maliciosos o errores de lógica afecten la infraestructura del servidor.
- JWT (JSON Web Token): Estándar abierto para la propagación de identidad de forma segura entre el cliente y el servidor, garantizando que el progreso de los puntos de experiencia (XP) pertenezca únicamente al usuario autenticado.
- Framer Motion: Biblioteca de animaciones para React utilizada para gestionar los movimientos de la interfaz, transiciones de niveles y feedback visual, mejorando el engagement del estudiante.
- REST API: Estilo de arquitectura de software que utiliza el protocolo HTTP para que el frontend de React se comunique de forma asíncrona con el backend de Node.js.
- ORM (Object-Relational Mapping): Técnica de programación para convertir datos entre el modelo de objetos de JavaScript y la base de datos relacional MySQL.
- Tailwind CSS: Framework de diseño basado en utilidades que permite construir interfaces personalizadas, responsivas y ligeras de forma eficiente.
- WCAG 2.1: Pautas de Accesibilidad para el Contenido Web, necesarias para garantizar que la plataforma sea inclusiva para todos los estudiantes, independientemente de sus capacidades técnicas o físicas.

1.4. Referencias

El desarrollo y validación de CodeBreaker se rigen por las siguientes normativas, estándares y fuentes técnicas:

- Estándar IEEE 830: Norma internacional para la elaboración de la Especificación de Requerimientos de Software (SRS), asegurando que los requerimientos funcionales y no funcionales estén correctamente definidos y documentados.
- ISO/IEC 25010: Modelo de calidad del producto de software, utilizado para evaluar la mantenibilidad, seguridad y eficiencia del sistema desarrollado bajo el entorno de ejecución de Node.js.

- Documentación Oficial de React.js: Referencia técnica para la implementación de la interfaz de usuario basada en componentes y la gestión del estado de la aplicación.
- Documentación de Node.js y Express: Guía oficial para el desarrollo de la lógica del servidor, la gestión de peticiones asíncronas y la arquitectura de la API REST.
- Referencia Técnica de Piston API: Documentación para la integración del motor de ejecución de código multilingüe de código abierto en entornos aislados.
- Guía de Estilos de Tailwind CSS: Estándar utilizado para el diseño de la interfaz responsiva, asegurando consistencia visual y optimización de carga.
- Pautas de Accesibilidad WCAG 2.1: Referencia fundamental para asegurar que la plataforma educativa sea accesible para todos los usuarios, cumpliendo con los estándares de inclusión digital.

1.5. Visión General

El proyecto consiste en el desarrollo de una plataforma web centralizada para el Aprendizaje Interactivo de Lenguajes de Programación (Python y PHP). A diferencia de un entorno de lectura convencional, este sistema implementa una arquitectura de Single Page Application (SPA) que garantiza una práctica fluida, segura y con progreso estructurado para cada estudiante.

La solución se articula en tres pilares fundamentales que definen su funcionamiento:

- Configuración y Control de Aprendizaje: El sistema permite la Gestión de Niveles (validación técnica de ejercicios) y la Gestión de Progreso (formalización del aprendizaje secuencial). Esta estructura asegura que el estudiante siga una ruta lógica, respetando un orden de dificultad y prerrequisitos validados en la base de datos MySQL antes de avanzar a retos complejos.
- Integridad mediante Ejecución Aislada (Piston API): Como regla de negocio crítica, el sistema delega la práctica a un entorno seguro y externo. El backend desarrollado en Node.js procesa las peticiones del Monaco Editor y las envía a la API de Piston, que ejecuta el código en un ambiente aislado. Cualquier error crítico o intento de ejecución maliciosa es contenido fuera del servidor principal, garantizando la integridad total de la infraestructura.
- Interacción y Gamificación Dinámica: El sistema mantiene alta la motivación del usuario mediante retroalimentación inmediata y visualmente atractiva. Esto incluye el Panel de Resultados con transiciones fluidas (Framer Motion) para visualizar errores en tiempo real, y un Sistema de Logros encargado de validar el cumplimiento de

retos y la acumulación de XP, incentivando la persistencia del estudiante de forma lúdica.

2. Posicionamiento

2.1. Oportunidad de negocio/educativo

En el ecosistema académico y profesional de Tacna, existe una demanda creciente por formación en desarrollo de software de alto rendimiento. Sin embargo, los estudiantes y entusiastas de la programación enfrentan barreras técnicas significativas: la configuración de entornos locales complejos, la disparidad de recursos de hardware y la falta de plataformas interactivas que permitan la práctica inmediata sin instalaciones previas.

La tendencia global hacia la educación técnica accesible y la transformación digital en las instituciones educativas peruanas representa una oportunidad estratégica. CodeBreaker surge para cerrar esta brecha, ofreciendo una solución que democratiza el aprendizaje de lenguajes clave como Python y PHP. Al utilizar una arquitectura moderna basada en Node.js para el procesamiento asíncrono y la API de Piston para la ejecución aislada de código, la plataforma no solo reduce los costos de infraestructura educativa, sino que garantiza un entorno de aprendizaje ágil, seguro y escalable, alineado con las exigencias del mercado tecnológico actual.

Aspecto	Descripción
Para	Estudiantes de ingeniería, entusiastas de la programación y centros educativos en Tacna.
Quiénes	Usuarios que buscan aprender lenguajes de programación de manera práctica y sin barreras técnicas iniciales (instalaciones o compiladores).
El problema de	El proceso tradicional de aprendizaje es lento debido a configuraciones de entorno complejas, hardware limitado y falta de retroalimentación inmediata.
La solución	Aplicación web SPA (React.js) con editor profesional integrado (Monaco Editor), ejecución segura en la nube vía Piston API y sistema de gamificación.
Que permite	Aprender a programar en Python/PHP de forma interactiva, visualizar el progreso mediante animaciones fluidas (Motion) y garantizar la ejecución de código en tiempo real sin riesgos.

2.2. Definición del problema

La situación actual en el aprendizaje de lenguajes de programación (enfocado en Python y PHP) presenta los siguientes problemas críticos identificados:

Problema	Impacto
Fricción técnica por configuración de entornos	Los estudiantes pierden tiempo excesivo instalando intérpretes, variables de entorno y extensiones, lo que genera frustración inicial y una alta tasa de abandono antes de escribir la primera línea de código.
Riesgos de seguridad en ejecución de código	Ejecutar scripts de estudiantes sin un entorno de Sandboxing (como la API de Piston) pone en riesgo la estabilidad del servidor ante bucles infinitos o comandos maliciosos.
Ausencia de feedback asíncrono e inmediato	La falta de una validación automática (basada en casos de prueba) obliga al estudiante a depender de revisiones manuales, ralentizando el ciclo de aprendizaje y la corrección de errores de lógica.
Bajo compromiso (Engagement) y gamificación	Al no existir una interfaz dinámica y animada (Motion) que recompense el progreso con niveles o XP, el aprendizaje se percibe como una tarea monótona en lugar de un reto interactivo.
Limitaciones por hardware heterogéneo	Los entornos de desarrollo (IDEs) pesados excluyen a estudiantes con equipos de bajos recursos. La falta de una SPA (React) ligera impide un aprendizaje fluido desde cualquier dispositivo.
Fragmentación del progreso académico	La inexistencia de una persistencia centralizada en la nube (MySQL) provoca que el avance del alumno se pierda al cambiar de terminal, dificultando el seguimiento docente.

3. Descripción de los interesados y usuarios

3.1. Resumen de los interesados

Nombre	Descripción	Responsabilidades
Comunidad Académica (Tacna)	Estudiantes de ingeniería y entusiastas de la tecnología de la región.	Usuarios finales que validarán la usabilidad, la fluidez de la interfaz (React) y la efectividad del aprendizaje autónomo.
Administradores de Infraestructura	Responsables de la gestión del servidor y servicios externos.	Supervisar la conectividad con la API de Piston, la salud del backend en Node.js y la integridad de la base de datos MySQL.
Equipo de Desarrollo (UPT)	Estudiantes de Ingeniería de Sistemas de la Universidad Privada de Tacna.	Diseñar, desarrollar (usando Stack basado en Node.js, React y MySQL), probar la integración del Monaco Editor y documentar el sistema.
Docente del Curso	Ing. Patrick Jose CUADROS QUIROGA.	Supervisar y evaluar el cumplimiento de los requerimientos, la aplicación de patrones de software y la calidad técnica del proyecto.

3.2. Resumen de los usuarios

Nombre	Descripción	Responsabilidades
Estudiante	Usuario que busca aprender Python o PHP.	Realizar ejercicios, ejecutar código en el editor y gestionar su progreso y logros.

Administrador de Contenido	Perfil técnico encargado de la plataforma.	Configurar nuevos niveles, retos de programación y validar las reglas de gamificación (XP).
Invitado / Visitante	Usuario no registrado.	Explorar la landing page y conocer las funcionalidades antes de crear una cuenta.

3.3. Entorno de usuario

Los usuarios acceden a la plataforma CodeBreaker mediante un navegador web moderno (Google Chrome, Mozilla Firefox, Microsoft Edge o Safari) con soporte para Web Workers y JavaScript avanzado. Dado que el sistema está diseñado para la educación accesible, no se requiere la instalación de software adicional en el equipo del estudiante (como compiladores de Python o intérpretes de PHP).

El sistema opera bajo el siguiente entorno tecnológico:

- Frontend (Interfaz de Usuario): Desarrollado como una SPA (Single Page Application) utilizando React.js. Se emplea Tailwind CSS para el diseño responsivo y Framer Motion para gestionar las animaciones y transiciones fluidas de la interfaz.
- Entorno de Desarrollo y Backend: Desarrollado íntegramente en Node.js con el framework Express, utilizando Visual Studio Code como entorno de desarrollo principal. Se emplea una arquitectura asíncrona para la gestión eficiente de peticiones y validaciones.
- Servidor de Aplicaciones y Lógica: Basado en el motor de ejecución V8 (Node.js), encargado de la orquestación de servicios, autenticación mediante JWT (JSON Web Tokens) y la gestión de la lógica de gamificación.
- Base de Datos: Relacional utilizando MySQL, encargada de la persistencia de perfiles de usuario, estructura de los niveles de aprendizaje, historial de XP y el registro de logros obtenidos.
- Motor de Ejecución (Sandbox): Integración directa con la API de Piston, que permite procesar el código de los estudiantes en entornos de ejecución aislados y seguros, devolviendo los resultados al backend en formato JSON.
- Editor de Código: Interfaz web profesional basada en Monaco Editor (el núcleo de VS Code), que proporciona resaltado de sintaxis, autocompletado y una experiencia de desarrollo industrial para Python y PHP.

- Compatibilidad de Dispositivos: Gracias al uso de Tailwind CSS, el sistema está optimizado para computadoras de escritorio, laptops y dispositivos móviles con resoluciones mínimas de 360px de ancho, garantizando una experiencia de aprendizaje ubicua.

3.4. Perfiles de los interesados

I. Autoridad Académica (Facultad de Ingeniería - UPT)

- Representante: Dirección de la Escuela Profesional de Ingeniería de Sistemas.
- Descripción: Máxima autoridad académica interesada en la implementación de herramientas tecnológicas que mejoren el aprendizaje de la programación en la facultad.
- Responsabilidad: Validar que la plataforma cumpla con los estándares educativos, supervisar el impacto en el aprendizaje y facilitar el acceso a la comunidad estudiantil.
- Criterio de éxito: Incremento en la tasa de finalización de ejercicios de programación y reducción de la brecha técnica en estudiantes de ciclos iniciales.
- Nivel de participación: Revisión de hitos académicos y validación del impacto educativo.

II. Administración de Infraestructura Tecnológica

- Representante: Responsables de Laboratorios o Servidores de la Facultad.
- Descripción: Personal técnico encargado de los recursos de hardware y conectividad donde se desplegará la plataforma.
- Responsabilidad: Supervisar el despliegue del entorno de Sandboxing (Docker), garantizar la disponibilidad de la JavaScript/TypeScript y asegurar la integridad de la base de datos del sistema.
- Criterio de éxito: Despliegue estable de la arquitectura en, con un aislamiento efectivo del Sandbox que no comprometa la red universitaria.
- Nivel de participación: Activo durante las fases de configuración del servidor, despliegue de contenedores y pruebas de carga.

III. Equipo de Desarrollo (UPT)

- Representante: Estudiantes de Ingeniería de Sistemas (Fuentes Avalos, Sebastian Nicolas y equipo).
- Descripción: Desarrolladores responsables de la construcción integral de la solución técnica.

- Responsabilidad: Diseñar la arquitectura en capas, programar el backend en, configurar el entorno en VS Code, implementar la gamificación y asegurar el cumplimiento de los requerimientos.
- Criterio de éxito: Entrega de una plataforma funcional que cumpla con los requerimientos de ejecución segura (RF-07) y gamificación (RF-05).
- Nivel de participación: Total y continuo durante todo el ciclo de vida del proyecto.

3.5. Perfiles de los Usuarios

Rol	Actividades principales	Frecuencia de uso
Administrador del Sistema	Gestión de cuentas de usuario, monitoreo del servidor Node.js, supervisión de la conectividad con la API de Piston y mantenimiento de la base de datos MySQL.	Semanal
Administrador de Contenido	Carga y configuración de retos de programación en la base de datos, definición de casos de prueba (entradas/salidas) y ajuste de los parámetros de XP y niveles.	Semanal / Quincenal
Estudiante (Usuario Final)	Resolución de retos usando el Monaco Editor, visualización de retroalimentación animada (Framer Motion), consulta de progreso y gestión de su perfil de aprendizaje.	Diaria
Invitado	Exploración de la plataforma a través de la landing page desarrollada en React, visualización de demostraciones y proceso de registro mediante JWT.	Ocasional

3.6. Necesidades de los interesados y usuarios

3.6.1. Necesidades

Nro°	Descripción de la Necesidad
N-01	Acceso y Seguridad de Identidad: El sistema debe garantizar un acceso seguro, protegiendo los datos del estudiante y manteniendo su sesión personalizada.
N-02	Entorno de Práctica Interactiva: El usuario requiere un espacio de trabajo con herramientas que faciliten la escritura de código y la visualización de resultados.
N-03	Seguridad y Aislamiento de Ejecución: Se necesita un entorno controlado (Sandbox) que proteja la infraestructura del servidor de errores o código malicioso.
N-04	Estructura de Contenido Académico: El sistema requiere rutas de aprendizaje organizadas para los lenguajes Python y PHP que guíen al estudiante desde lo básico.
N-05	Gestión y Mantenibilidad del Sistema: Los administradores necesitan una arquitectura modular que permita escalar el sistema y actualizar contenidos sin afectar el núcleo en Node.js.
N-06	Inclusión y Diversidad Pedagógica: La plataforma debe ofrecer diversos tipos de retos y ser accesible para usuarios con diferentes capacidades.
N-07	Eficiencia y Escalabilidad: El sistema debe responder rápidamente y soportar múltiples usuarios concurrentes sin degradar el rendimiento.
N-08	Validación y Tutoría Automática: Se requiere que el sistema evalúe el código de forma autónoma y brinde retroalimentación precisa al estudiante.
N-09	Trazabilidad y Evaluación: El sistema debe registrar el historial de intentos y errores para permitir el seguimiento del aprendizaje.
N-10	Motivación y Gamificación: El usuario necesita incentivos visuales y recompensas (XP, logros) que fomenten la persistencia en el estudio.
N-11	Ubicuidad y Experiencia de Usuario: La interfaz debe ser intuitiva y adaptarse perfectamente tanto a computadoras como a dispositivos móviles Android.

3.6.2. Requerimientos

N°	Requerimientos	Descripción	Necesidad
RF-01	Registro de usuario	El sistema debe permitir registrar nuevos usuarios mediante correo electrónico y contraseña, o mediante proveedor OAuth (Google/GitHub).	N-01
RF-02	Inicio de sesión	El sistema debe autenticar usuarios registrados y mantener la sesión activa mediante tokens JWT o equivalente.	N-01
RF-03	Cierre de sesión	El usuario debe poder cerrar sesión en cualquier momento, invalidando el token activo.	N-01
RF-04	Perfil de usuario	El sistema debe mostrar al usuario su nombre, avatar, nivel actual, XP acumulado y logros obtenidos.	N-01
RF-05	Guardado de progreso	El progreso del usuario (niveles completados, XP, logros) debe persistir en base de datos y sincronizarse entre dispositivos.	N-10
RF-06	Editor integrado	El sistema debe incluir un editor de código con resaltado de sintaxis, autocompletado básico e indentación automática para Python y PHP.	N-02
RF-07	Ejecución en sandbox	El usuario debe poder ejecutar su código mediante una API segura (Piston o Judge0), sin afectar el servidor principal.	N-03
RF-08	Visualización de resultados	El sistema debe mostrar la salida estándar (stdout) y los errores (stderr) del código ejecutado en un panel de resultados.	N-02
RF-09	Límite de ejecución	Cada ejecución no debe superar los 10 segundos de cómputo ni 256 MB de memoria para evitar abuso del sandbox.	N-03
RF-10	Soporte multiformato	El editor debe soportar al menos Python 3.x y PHP 8.x como lenguajes de ejecución.	N-02
RF-11	Ruta Python	El sistema debe ofrecer al menos 10 niveles iniciales para Python: variables, tipos de datos, condicionales, bucles, funciones y listas.	N-04
RF-12	Ruta PHP	El sistema debe ofrecer al menos 10 niveles iniciales para PHP: sintaxis básica, variables, arrays, funciones y formularios.	N-04
RF-13	Tipos de ejercicio	Cada nivel debe incluir al menos un tipo: completar código, corregir errores, escribir desde cero u opción múltiple.	N-06
RF-14	Instrucciones de nivel	Cada nivel debe mostrar descripción del objetivo, ejemplos y resultado esperado antes del ejercicio.	N-08

RF-15	Progresión bloqueada	Los niveles deben desbloquearse secuencialmente; el usuario no puede acceder a un nivel sin completar el anterior.	N-04
RF-16	Validación automática	El sistema debe comparar la salida del usuario con la salida esperada para determinar si el nivel fue completado.	N-08
RF-17	Retroalimentación inmediata	Al fallar un ejercicio, el sistema debe indicar qué parte es incorrecta y ofrecer una pista o mensaje de error descriptivo.	N-08
RF-18	Sistema de XP y niveles	El usuario debe ganar XP al completar niveles; al acumular suficiente XP sube de nivel de perfil.	N-10
RF-19	Sistema de logros	El sistema debe otorgar insignias por hitos: primer nivel completado, completar una ruta entera, racha de 7 días, etc.	N-10
RF-20	Diseño responsivo	La interfaz debe adaptarse a escritorio (1024px+) y dispositivos móviles Android (360px+).	N-11
RF-21	Mapa de progreso	Debe existir una pantalla visual con el mapa de niveles disponibles, completados y bloqueados para cada ruta.	N-11
RF-22	Navegación táctil	Los controles clave (ejecutar, avanzar nivel, ver pistas) deben ser accesibles con tap sin necesidad de hover.	N-11

3.6.3. Requerimientos NO Funcionales

N°	Requerimientos	Descripción	Necesidad
RNF-01	Tiempo de carga inicial	La aplicación debe cargar su página principal en menos de 3 segundos en una conexión de 10 Mbps.	N-07
RNF-02	Latencia de ejecución	El resultado de la ejecución de código debe mostrarse en menos de 5 segundos en condiciones normales de red.	N-07
RNF-03	Respuesta del editor	El editor no debe presentar retraso perceptible (>100 ms) al tipear en dispositivos de gama media-alta.	N-02
RNF-04	Carga de niveles	La transición entre niveles debe completarse en menos de 1 segundo después de la validación.	N-07
RNF-05	Soporte de concurrencia	El backend debe soportar al menos 50 usuarios concurrentes ejecutando código sin degradación del servicio.	N-07
RNF-06	Aislamiento de ejecución	El código del usuario debe ejecutarse en sandbox aislado, sin acceso al sistema de archivos ni red del servidor.	N-03

RNF-07	Protección de contraseñas	Las contraseñas deben almacenarse con hash bcrypt (cost factor >= 10); nunca en texto plano.	N-01
RNF-08	Comunicación cifrada	Toda comunicación entre cliente y servidor debe realizarse sobre HTTPS/TLS 1.2+.	N-01
RNF-09	Validación de entradas	El sistema debe sanitizar todas las entradas del usuario para prevenir inyección SQL, XSS y ejecución maliciosa.	N-03
RNF-10	Onboarding intuitivo	Un usuario sin experiencia debe poder iniciar su primer ejercicio en menos de 3 minutos desde el registro.	N-11
RNF-11	Accesibilidad básica	La interfaz debe cumplir nivel AA de WCAG 2.1: contraste adecuado, textos alternativos y navegación por teclado.	N-06
RNF-12	Mensajes de error claros	Los errores deben presentarse en lenguaje comprensible para principiantes, evitando jerga técnica interna.	N-02
RNF-13	Consistencia visual	Los componentes de UI deben mantener un sistema de diseño coherente en todas las pantallas.	N-11
RNF-14	Arquitectura modular	El backend debe separar responsabilidades en módulos independientes (auth, ejecución, progreso, contenido).	N-05
RNF-15	Contenido configurable	Los niveles deben definirse en archivos de configuración o BD, permitiendo agregar contenido sin modificar el código.	N-05
RNF-16	Escalabilidad horizontal	La arquitectura debe permitir agregar instancias del servicio de ejecución de código para escalar ante mayor demanda.	N-07
RNF-17	Disponibilidad del servicio	La plataforma debe apuntar a 99% de disponibilidad durante el período académico.	N-11
RNF-18	Compatibilidad de navegadores	La app debe funcionar en Chrome 110+, Firefox 110+, Edge 110+ y Safari 15+ en escritorio y móvil.	N-11
RNF-19	Compatibilidad Android	La experiencia móvil debe estar optimizada para Android 10+, desde 360x640px, con teclado virtual y gestos táctiles.	N-06

4. Vista General del Producto

4.1. Perspectiva del producto

CodeBreaker es una aplicación web de tipo SPA (Single Page Application) diseñada para la formación técnica interactiva. Su objetivo principal es actuar como una plataforma de aprendizaje guiado que elimina las barreras de

configuración inicial, permitiendo que el estudiante comience a programar de inmediato sin instalar compiladores o entornos locales.

El sistema se articula mediante un stack tecnológico moderno basado en JavaScript (React.js y Node.js) y una arquitectura de servicios que orquestan las siguientes fases:

- Entorno de Práctica: Interfaz de usuario dinámica desarrollada con React.js y estilizada con Tailwind CSS. Integra el motor Monaco Editor para ofrecer una experiencia de escritura de código profesional (similar a VS Code) y utiliza Framer Motion para proporcionar una navegación fluida y animada.
- Motor de Ejecución Externa (Piston API): El backend, construido en Node.js, actúa como un puente seguro que delega la ejecución del código a la API de Piston. Esto permite procesar lenguajes como Python y PHP en contenedores de alto rendimiento totalmente aislados del servidor principal.
- Capa de Validación y Gamificación: Módulo lógico que procesa las respuestas en formato JSON, comparando los resultados obtenidos contra casos de prueba predefinidos en una base de datos MySQL. Gestiona la progresión de niveles, el desbloqueo de retos y la asignación de puntos de experiencia (XP) en tiempo real.

4.2. Resumen de capacidades

Capacidad	Beneficio para el interesado
Interfaz SPA de alto rendimiento (React.js)	Proporciona una navegación instantánea y fluida, mejorando la experiencia de usuario (UX) frente a sistemas tradicionales.
Motor de edición profesional (Monaco Editor)	Ofrece al estudiante herramientas industriales como resaltado de sintaxis y autocompletado, idénticas a las de Visual Studio Code.
Ejecución multilingüe aislada (Piston API)	Permite procesar código en Python y PHP de forma segura y ultra rápida, sin comprometer los recursos del servidor local.
Feedback visual animado (Framer Motion)	Utiliza transiciones y movimientos fluidos en la interfaz para reforzar la retroalimentación positiva y el compromiso del alumno.
Diseño responsivo y ligero (Tailwind CSS)	Asegura una visualización perfecta y tiempos de carga mínimos tanto en PCs de laboratorio como en smartphones.
Validación automática de ejercicios (Node.js)	Compara en tiempo real la salida del código con los resultados esperados, permitiendo un aprendizaje autónomo y dinámico.
Sistema de Gamificación (XP y Logros)	Incrementa la retención mediante recompensas visuales y un sistema de niveles basado en el progreso almacenado en MySQL.
Gestión de niveles con progresión bloqueada	Estructura el aprendizaje de forma lógica, asegurando que el estudiante domine los fundamentos antes de avanzar.
Historial de trazabilidad y métricas	Registra cada intento y error del alumno, proporcionando datos valiosos para que el docente evalúe el desempeño real.
Seguridad de sesiones mediante JWT	Protege la integridad de los datos de los estudiantes y el acceso administrativo mediante tokens de seguridad modernos.

4.3. Suposiciones y dependencias

4.3.1. Suposiciones

- Los usuarios (estudiantes y administradores) disponen de un dispositivo con acceso a internet y un navegador web moderno que soporte Web Workers, necesarios para el funcionamiento óptimo de Monaco Editor.

- El servidor de aplicaciones tiene instalado el entorno de ejecución Node.js (versión LTS) y el gestor de paquetes npm o yarn para la gestión del backend.
- El entorno de desarrollo en Visual Studio Code cuenta con las extensiones de ESLint, Prettier y Tailwind CSS IntelliSense para asegurar la calidad del código y la rapidez en el diseño.
- Se asume que el servidor de despliegue permite la salida de peticiones hacia puertos externos para la comunicación con la API de Piston.
- La interfaz de usuario mantendrá su fluidez y animaciones (Framer Motion) siempre que el hardware del cliente disponga de aceleración por hardware básica en el navegador.

4.3.2. Dependencias

- Disponibilidad de Piston API: El núcleo funcional del sistema depende estrictamente de la disponibilidad y tiempo de respuesta de los servidores de Piston para la ejecución de los scripts de Python y PHP.
- Infraestructura de Base de Datos MySQL: El sistema requiere acceso continuo y latencia baja a un servidor de base de datos MySQL para la persistencia del progreso, cálculo de XP y almacenamiento de retos.
- Ecosistema de Paquetes npm: Dependencia de la disponibilidad de repositorios oficiales para la instalación y actualización de bibliotecas críticas como React, Express, Sequelize y Framer Motion.
- Servicios de Identidad (JWT): Dependencia de la correcta configuración de llaves secretas y algoritmos de cifrado en el servidor Node.js para la emisión y validación de JSON Web Tokens.
- Conectividad para Recursos de Frontend: Dependencia de acceso a redes de distribución de contenido (CDN) para la carga de fuentes tipográficas y librerías de estilos de Tailwind CSS.

4.4. Costos y precios

4.4.1. Costos Generales

Esta tabla detalla la inversión en hardware necesaria para establecer el entorno de desarrollo y pruebas locales del sistema en.

Recurso	Cantidad	Costo
Computadora	4 unidad	S/. 9,600
Teclado y Mouse	4 sets	S/. 400
Total		S/. 10,000

4.4.2. Costos operativos durante el desarrollo

Gastos recurrentes para mantener la infraestructura base durante los hitos de abril, mayo y junio.

Concepto	Costo Mensual	Duración	Costo Total
Servicios básicos (Luz, agua, internet de alta velocidad)	S/. 400	3 meses	S/. 1,200
Soporte técnico y licencias (Suscripciones de software o soporte especializado)	S/. 200	3 meses	S/. 600
Renta de espacio de trabajo (Valorizado del lugar donde se reúnen a programar)	S/. 800	3 meses	S/. 2,400
TOTAL OPERATIVOS			S/. 4,200

4.4.3. Costos del ambiente

Costo de mantenimiento del sistema "vivo" en la nube para el acceso de los estudiantes.

Concepto	Duración	Costo (S/.)
Dominio Web (Ejemplo: firma.zofratacna.gob.pe)	12 meses	S/. 75
Servidor en la nube (Render/Railway para el backend y BD SQL)	12 meses	S/. 225

TOTAL AMBIENTE		S/. 300

4.4.4. Costos de personal

Basado en el equipo de desarrollo del proyecto CodeBreaker.

Rol	Integrante	Costo Mensual	Total (3 meses)
Líder de Proyecto / Analista	Gutierrez Mamani, Gabriela Luzkalid	S/. 1,000	S/. 3,000
Desarrollador Backend	Rosas Chambilla, Jefferson	S/. 850	S/. 2,550
Desarrollador Frontend	Fuentes Avalos, Sebastian Nicolas	S/. 850	S/. 2,550
Tester / QA	Chire Ramos, Mayra Fernanda	S/. 600	S/. 1,800
TOTAL PERSONAL			S/. 9,900

4.4.5. Costos totales del desarrollo del sistema

Sumando todos los costos involucrados en el desarrollo, el costo total estimado es el siguiente:

Concepto	Costo en Soles
Costos generales	S/. 10,000
Costos operativos durante el desarrollo	S/. 4,200
Costos del ambiente	S/. 300
Costos de personal	S/. 9,900
Costos totales	S/. 24,400

4.5. Licenciamiento e instalación

4.5.1. Licenciamiento

El sistema CodeBreaker se desarrolla bajo un modelo de Software Libre (ej. Licencia MIT o Apache 2.0), lo que permite a la institución

educativa utilizar, modificar y distribuir la plataforma sin costos de licenciamiento de software propietario.

El stack tecnológico se basa íntegramente en componentes de código abierto:

- Lenguaje: JavaScript / Node.js
- Entorno de Ejecución: Node.js (Runtime) y Docker (para el Sandboxing de ejecución de código).
- Base de Datos: MySQL (Community Server).
- IDE de Desarrollo: Visual Studio Code con extensiones de código abierto.

4.5.2. Instalación y Despliegue

La instalación se realizará siguiendo un flujo de integración continua para garantizar la disponibilidad del sistema:

Configuración del Servidor: El despliegue se llevará a cabo en servidores con soporte para la JavaScript/TypeScript . Se recomienda el uso de entornos Linux (Ubuntu Server o Debian) por su eficiencia en el manejo de contenedores.

Entorno de Sandboxing: Se requiere la configuración de un motor de Docker o la conexión mediante llaves de API a servicios externos de ejecución segura (como Piston), asegurando que los límites de memoria y tiempo (RNF-04) estén correctamente parametrizados.

Persistencia de Datos: Se ejecutará el script de migración de base de datos para establecer las tablas de usuarios, niveles y logros, garantizando la integridad referencial desde el inicio.

Seguridad de Red: Se deberá instalar un certificado SSL/TLS (vía Let's Encrypt o similar) para habilitar el protocolo HTTPS, cumpliendo con el requerimiento de comunicación cifrada (RNF-08).

5. Características del producto

- Gestión de Retos y Contenido: El sistema permitirá al Administrador de Contenido cargar ejercicios de programación en Python y PHP, definiendo metadatos (título, lenguaje, dificultad), instrucciones detalladas y los casos de prueba (inputs/outputs) para la validación.
- Editor de Código Interactivo: La plataforma contará con un editor integrado basado en web que permitirá a los estudiantes escribir código con resaltado de sintaxis, autocompletado e indentación automática, eliminando la necesidad de instalar IDEs locales.
- Ejecución en Sandbox Seguro: El sistema procesará el código de los estudiantes en un entorno aislado mediante contenedores (Docker), garantizando que errores como bucles infinitos o scripts maliciosos no afecten la integridad del servidor principal.

- Validación Automática de Resultados: El backend desarrollado en JavaScript / Node.js comparará automáticamente la salida del código del usuario con los resultados esperados. Solo si coinciden, el ejercicio se marcará como "Completado".
- Retroalimentación Inmediata: Al detectar errores de sintaxis o de lógica, el sistema mostrará un panel de resultados con mensajes descriptivos y pistas para ayudar al estudiante a corregir su código de forma autónoma.
- Sistema de Progresión Secuencial: Los niveles de aprendizaje estarán bloqueados jerárquicamente. El estudiante debe completar con éxito los retos base (ej. variables) antes de que el sistema habilite automáticamente temas avanzados (ej. funciones).
- Gamificación (XP y Logros): Por cada reto superado, el usuario acumulará puntos de experiencia (XP) y podrá desbloquear insignias o logros, incentivando la persistencia y competitividad sana en el aprendizaje.
- Mapa de Progreso en Tiempo Real: Los usuarios dispondrán de una interfaz visual que muestra su avance actual en la ruta de aprendizaje, identificando niveles superados, retos pendientes y su posición en el ranking de niveles.
- Trazabilidad y Auditoría de Aprendizaje: El sistema registrará el historial detallado de cada intento de ejecución, incluyendo el código enviado, errores recurrentes y tiempos de resolución, sirviendo como evidencia de progreso para el docente.
- Diseño Responsivo y Navegación Táctil: La interfaz está optimizada para funcionar tanto en computadoras de laboratorio como en dispositivos móviles Android, con controles adaptados para el uso de teclados virtuales y gestos táctiles.
- Bandejas de Actividad Organizadas: Los estudiantes verán sus tareas organizadas en categorías como "Por empezar", "En curso" y "Completados", facilitando la gestión de sus sesiones de estudio.
- Control de Acceso Seguro (JWT): La gestión de identidades y sesiones se realizará mediante tokens seguros (JSON Web Tokens), definiendo roles de Administrador, Estudiante e Invitado para proteger la integridad de los datos de progreso.
- Notificaciones de Hitos: El sistema enviará alertas visuales y notificaciones internas cuando el usuario desbloquee una nueva ruta de aprendizaje o alcance un nivel de perfil superior.

6. Restricciones

N°	Restricción
R-01	El sistema debe desarrollarse utilizando el stack de JavaScript: React.js para el frontend y Node.js (Express) para el backend, centralizando el desarrollo en Visual Studio Code.
R-02	El gestor de base de datos obligatorio es MySQL, asegurando que el diseño de las tablas sea compatible con el almacenamiento de estructuras JSON para los casos de prueba.
R-03	La ejecución de código (Python/PHP) debe delegarse exclusivamente a la API de Piston, garantizando un entorno de ejecución aislado y seguro fuera del servidor de la aplicación.
R-04	La interfaz de usuario debe implementarse utilizando Tailwind CSS para asegurar un diseño responsivo y el cumplimiento de estándares de rendimiento visual.
R-05	Las transiciones y dinámicas de la interfaz deben ser gestionadas por librerías de movimiento (Framer Motion), garantizando una experiencia de usuario fluida y moderna.
R-06	El motor de edición de código integrado debe ser Monaco Editor, configurado para restringir la entrada de archivos que no correspondan a las extensiones permitidas (.py, .php).
R-07	El flujo de aprendizaje es estrictamente secuencial; el backend en Node.js bloqueará el acceso a rutas avanzadas si no existe el registro de éxito del nivel anterior en MySQL.
R-08	La validación de retos es automática: el sistema comparará los strings de salida capturados de la API de Piston contra el resultado esperado, sin permitir la asignación manual de XP.
R-09	Toda comunicación de datos entre el frontend (React) y el backend (Node.js) debe realizarse bajo el protocolo HTTPS y el formato de intercambio de datos JSON.
R-10	La gestión de identidades y persistencia de sesión se realizará mediante JSON Web Tokens (JWT), con tiempos de expiración controlados para proteger el perfil del estudiante.

7. Rangos de calidad

- Seguridad y Aislamiento: El sistema garantizará que la ejecución de código (Python/PHP) sea totalmente aislada mediante Sandboxing (Docker), impidiendo accesos no autorizados al servidor. La comunicación será cifrada vía HTTPS y la autenticación se gestionará mediante estándares JWT.
- Integridad de los Datos: El sistema asegurará que el progreso (XP, niveles y logros) y el historial de trazabilidad sean inalterables. Cualquier intento de manipulación externa de los resultados de validación será bloqueado por la lógica del backend en JavaScript / Node.js .
- Disponibilidad y Rendimiento: La plataforma mantendrá una disponibilidad del 99% durante el ciclo académico. Los tiempos de respuesta para la validación de código no excederán los 5 segundos, garantizando una experiencia de aprendizaje fluida sin latencias críticas.
- Compatibilidad Multiplataforma: La solución será totalmente funcional en navegadores modernos (Chrome, Firefox, Edge, Safari) y estará optimizada para dispositivos móviles Android, cumpliendo con los estándares de diseño responsivo.
- Usabilidad y Accesibilidad: La interfaz será intuitiva (Onboarding < 3 min), permitiendo que estudiantes sin experiencia previa en IDEs complejos puedan empezar a programar de inmediato. Se cumplirá con el nivel AA de la norma WCAG 2.1 para garantizar la inclusión.

8. Precedencia y Prioridad

- Gestión de Identidad y Acceso (JWT): Es la prioridad máxima y el punto de entrada. Garantiza que el progreso (XP) y los logros se asignen al usuario correcto y protege las funciones administrativas de carga de contenido.
- Entorno de Edición y Sandbox (Java/Docker): Constituye el núcleo técnico. No se puede avanzar sin un editor funcional y un motor de ejecución seguro que aísle el código del estudiante para proteger el servidor.
- Carga y Configuración de Retos: Representa la base del flujo educativo. Define los objetivos, lenguajes (Python/PHP) y, lo más importante, los casos de prueba que rigen la lógica del sistema.
- Validación Automática de Resultados: Es la fase funcional crítica. El backend en JavaScript / Node.js debe comparar salidas en milisegundos para determinar si el estudiante ha cumplido con los objetivos pedagógicos.
- Progresión y Bloqueo Secuencial: Marca la lógica del aprendizaje. Asegura que el usuario no acceda a niveles complejos sin haber superado los retos fundamentales, garantizando una curva de aprendizaje adecuada.

- Sistema de Gamificación (XP y Logros): Actúa como el motor de retención. Una vez validado el ejercicio, el sistema actualiza inmediatamente el perfil, otorgando recompensas visuales que incentivan la continuidad.
- Trazabilidad y Monitoreo de Intentos: Acompaña todo el proceso. Registra cada error y éxito para generar el historial de aprendizaje que servirá de evidencia tanto para el alumno como para el docente.
- Mapa de Progreso y Dashboard: Es el resultado visual final. Permite al usuario tener una visión holística de su avance y de las rutas de aprendizaje completadas y pendientes.
- Interfaz Responsiva y Accesibilidad: Garantiza que todas las funcionalidades anteriores sean utilizables en cualquier dispositivo (PC/Android), cumpliendo con los estándares de inclusión.

9. Otros requerimientos del producto

a. Estándares legales

- Ley N° 29733 – Ley de Protección de Datos Personales (Perú): El sistema debe garantizar que el tratamiento de los datos de los estudiantes (nombres, correos, progreso académico) se realice bajo los principios de seguridad y confidencialidad, implementando las medidas técnicas necesarias en el backend de JavaScript / Node.js para evitar accesos no autorizados.
- Decreto Legislativo N° 822 – Ley sobre el Derecho de Autor: La plataforma debe respetar los derechos de propiedad intelectual sobre el contenido educativo y el código fuente desarrollado. Asimismo, el sistema garantizará que los ejercicios propuestos no infrinjan derechos de terceros.
- Ley N° 30096 – Ley de Delitos Informáticos: El sistema debe implementar mecanismos de seguridad (como el Sandboxing) para prevenir el uso indebido de la infraestructura, asegurando que los usuarios no puedan realizar acciones que comprometan la integridad de sistemas externos o del propio servidor.
- Estándares de Accesibilidad (WCAG 2.1): En cumplimiento con las políticas de inclusión digital, la interfaz debe cumplir con los requisitos mínimos de accesibilidad para asegurar que estudiantes con discapacidad puedan navegar y aprender en la plataforma sin barreras técnicas.
- Normativa de Seguridad Digital (Seguimiento de lineamientos de la Secretaría de Gobierno Digital): Aunque el sistema es educativo, se seguirán las buenas prácticas de autenticación y cifrado (HTTPS/TLS) recomendadas para servicios digitales en el Perú, asegurando una plataforma robusta y confiable.

b. Estándares de comunicación

- Protocolo de Transferencia Segura: Todas las comunicaciones entre el cliente (React SPA) y el servidor backend (Node.js) deben realizarse obligatoriamente mediante el protocolo HTTPS, utilizando TLS 1.2 o superior. Esto garantiza el cifrado de extremo a extremo y la integridad de los datos académicos y personales transmitidos.

- Gestión de Sesiones y Autorización (JWT): El intercambio de información de identidad se realizará mediante JSON Web Tokens (JWT). Estos tokens serán gestionados en el cliente y enviados en las cabeceras de cada petición asíncrona (Authorization: Bearer), permitiendo una arquitectura stateless que facilita la escalabilidad del backend en Node.js.

- Integración con el Motor de Ejecución (Piston API): La comunicación con el servicio de ejecución de código se realizará mediante peticiones RESTful asíncronas hacia los endpoints de la API de Piston. Se implementará un sistema de gestión de errores y timeouts en Node.js para asegurar que el usuario reciba una respuesta (éxito o error de sintaxis) en menos de 5 segundos.

- Intercambio de Datos en Tiempo Real: Para las notificaciones de logros y actualizaciones de la interfaz sin recargar la página, se utilizarán tecnologías de comunicación bidireccional o el estado global de la aplicación en React, asegurando que los movimientos de la interfaz (Framer Motion) se activen instantáneamente tras recibir la respuesta del servidor.

- Estandarización de Datos (JSON): Toda la transferencia de información (estructuras de retos, código fuente enviado, resultados de compilación y estadísticas de XP) se realizará exclusivamente en formato JSON. Esto asegura una interoperabilidad total entre el frontend, el backend y la base de datos MySQL.

c. Estándares de cumplimiento de la plataforma

- Compatibilidad de Navegadores: El sistema debe garantizar un funcionamiento óptimo y una visualización consistente en las versiones actuales de Google Chrome, Mozilla Firefox, Microsoft Edge y Safari, asegurando que el editor de código integrado (basado en Monaco Editor o similar) mantenga su fluidez y resaltado de sintaxis.

- Estándar de Ejecución Segura (Sandboxing): El motor de validación debe cumplir con los protocolos de aislamiento de procesos. La plataforma debe ser capaz de reconocer y gestionar correctamente las salidas de error (stderr) y éxito (stdout) de los intérpretes de Python 3.x y PHP 8.x, validando los resultados sin comprometer la estabilidad del sistema operativo anfitrión.

- Compatibilidad de Base de Datos: El diseño del esquema y las consultas en el backend de JavaScript / Node.js deben seguir el estándar SQL ANSI, garantizando la portabilidad entre diferentes motores de bases de datos relacionales (como MySQL) y asegurando la integridad referencial de los perfiles de usuario y su progreso.

- Arquitectura de Software: El sistema debe seguir un patrón de arquitectura Limpia (Clean Architecture) o Arquitectura en Capas (Controladores, Servicios, Repositorios) desarrollada íntegramente en JavaScript / Node.js . Esto garantiza la separación de responsabilidades, facilitando las pruebas unitarias y la mantenibilidad del código desde el entorno de Visual Studio Code.

d. Estándares de calidad y seguridad

- Control de Acceso Basado en Roles (RBAC): El sistema debe implementar un control de acceso estricto mediante JSON Web Tokens (JWT), garantizando que solo los usuarios autenticados accedan a sus perfiles y que únicamente el rol "Administrador de Contenido" pueda modificar los retos y casos de prueba en el backend de JavaScript / Node.js .

- Trazabilidad y Auditoría de Aprendizaje: Todas las ejecuciones de código y cambios de nivel deben quedar registrados en logs de auditoría persistentes, capturando la fecha, hora, ID de usuario, código enviado y resultado de la validación, permitiendo un seguimiento detallado del progreso académico.

- Seguridad de Ejecución (Sandboxing): El sistema debe validar y aislar cada envío de código en un contenedor Docker independiente antes de procesarlo. Se deben aplicar restricciones de recursos (CPU y Memoria) para prevenir ataques de denegación de servicio (DoS) o fugas de memoria en el servidor host.

- Integridad de Resultados y Logros: El sistema garantizará que los puntos de experiencia (XP) y las insignias obtenidas sean inalterables mediante validaciones del lado del servidor (server-side). El estado de un ejercicio como "Completado" solo podrá ser otorgado por el motor de validación tras superar el 100% de los casos de prueba.

- Control de Concurrencia en el Sandbox: Se deben implementar mecanismos de gestión de colas para controlar el acceso simultáneo al motor de ejecución de código. El sistema limitará el número de ejecuciones concurrentes por usuario para asegurar un tiempo de respuesta óptimo (RNF-02) y evitar la saturación de los hilos de la JavaScript/TypeScript .

CONCLUSIONES

La plataforma educativa CodeBreaker representa una solución innovadora y robusta que responde directamente a las barreras de entrada identificadas en el aprendizaje de la programación. Al eliminar la necesidad de configuraciones locales complejas y centralizar el proceso en un entorno web, el sistema democratiza el acceso a la formación técnica en lenguajes críticos como Python y PHP.

La arquitectura del sistema, basada en un backend sólido en JavaScript / Node.js y una infraestructura de Sandboxing (Docker), garantiza que la experimentación de los estudiantes se realice en un entorno de ejecución seguro y controlado. Este enfoque no solo protege la integridad del servidor ante posibles errores de lógica o scripts maliciosos, sino que también proporciona una validación automática y precisa que actúa como un tutor constante para el alumno.

Finalmente, la integración de mecánicas de gamificación (XP, niveles y logros) y la trazabilidad detallada de los intentos de resolución aseguran un compromiso sostenido del usuario con su propio aprendizaje. CodeBreaker no es solo un editor de código, sino un ecosistema integral que proporciona a la institución una herramienta auditable y escalable para la formación de futuros ingenieros, cumpliendo con los más altos estándares de seguridad digital y protección de datos en el Perú.

RECOMENDACIONES

- Pruebas de Latencia con Piston API: Se recomienda realizar pruebas de estrés en la integración con el motor de ejecución externo desde las fases iniciales, para ajustar los tiempos de espera (timeouts) y asegurar que la respuesta del código sea fluida para el estudiante.
- Optimización del Estado en React: Se sugiere implementar una gestión de estado eficiente (usando Hooks o Context) para evitar que el renderizado de las animaciones de Framer Motion y el Monaco Editor saturen la memoria del navegador en dispositivos de bajos recursos.
- Gestión Dinámica de Retos: Se recomienda diseñar el módulo de carga de niveles de forma que los casos de prueba (entradas/salidas) se almacenen directamente en MySQL, permitiendo que el administrador agregue nuevos retos de Python o PHP en tiempo real sin necesidad de reiniciar el servidor Node.js.
- Escalabilidad de la Persistencia: Se sugiere diseñar la base de datos con índices optimizados para el historial de intentos y XP, anticipando un crecimiento en la trazabilidad conforme aumente el número de usuarios y la complejidad de los niveles de aprendizaje.
- Validación de Usabilidad Pedagógica: Se recomienda involucrar a estudiantes de ciclos iniciales en las pruebas de aceptación (UAT) para validar que los mensajes de error devueltos por la API y las pistas visuales de la interfaz sean comprensibles y faciliten el aprendizaje autónomo.
- Documentación de Arquitectura MERN: Se recomienda mantener una documentación exhaustiva en Visual Studio Code, incluyendo la configuración de variables de entorno (.env) y las dependencias de npm, facilitando el mantenimiento futuro y la escalabilidad del sistema.

BIBLIOGRAFÍA

IEEE Std 830-1998. IEEE Recommended Practice for Software Requirements Specifications. New York: Institute of Electrical and Electronics Engineers, 1998.

ISO/IEC 25010:2011. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. Geneva: International Organization for Standardization, 2011.

World Wide Web Consortium (W3C). Web Content Accessibility Guidelines (WCAG) 2.1. W3C Recommendation, 5 June 2018 (actualizado el 6 de mayo de 2025).

Open Web Application Security Project (OWASP). OWASP Top Ten 2025. 2025.

Internet Engineering Task Force (IETF). RFC 7519: JSON Web Token (JWT). Mayo de 2015.

WEBGRAFÍA

React. React – A JavaScript library for building user interfaces. Disponible en: https://react.dev/ [Consulta: 10 de abril de 2026].

Node.js. Node.js Documentation. Disponible en: https://nodejs.org/docs/latest/api/ [Consulta: 10 de abril de 2026].

Express. Express – Node.js web application framework. Disponible en: https://expressjs.com/ [Consulta: 10 de abril de 2026].

Piston. Piston API Documentation (v2). Disponible en: https://piston.readthedocs.io/en/latest/api-v2/ y https://github.com/engineer-man/piston [Consulta: 10 de abril de 2026].

Tailwind CSS. Tailwind CSS Documentation. Disponible en: https://tailwindcss.com/docs [Consulta: 10 de abril de 2026].

W3C. Web Content Accessibility Guidelines (WCAG) 2.1. Disponible en: https://www.w3.org/TR/WCAG21/ [Consulta: 10 de abril de 2026].

Open Web Application Security Project (OWASP). OWASP Top Ten 2025. Disponible en: https://owasp.org/Top10/2025/ [Consulta: 10 de abril de 2026].

Internet Engineering Task Force (IETF). RFC 7519: JSON Web Token (JWT). Disponible en: https://datatracker.ietf.org/doc/html/rfc7519 [Consulta: 10 de abril de 2026].